

□词的独热表示

□词的分布式表示

文本在计算机内部的存储和计算问题？

□ 字符串

□ 向量表示

文本的表示

- ❑ 字符串：字符序列，一个字符本质上就是一个整数

- ❑ 西文字符：一个字节存放

- ❑ 7位：ASCII码

- ❑ 最高位：0

- ❑ 比如：“A” 01000001

- ❑ 中文字符：二个字节存放一个中文字符

- ❑ 每个字节的最高位均为1

- ❑ “大” 的机内码是10110100 11110011

- ❑ 优点

- ❑ 文本最自然，最常用的机内存储形式

- ❑ 简单地实现字符串增删改查等编辑任务

- ❑ 通过编辑距离等算法计算两个字符串之间的字面相似度

文本表示-字符串

❑缺点

- ❑计算文本的语义信息时，往往需要使用基于规则的方法
 - ❑例如，要判断一个句子的情感极性（褒义或贬义）
 - ❑规则的形式可能为：如果句子中出现“喜欢”“漂亮”等词则为褒义；如果出现“讨厌”“丑陋”等词则为贬义
- ❑基于规则的方法存在问题
 - ❑规则的归纳依赖专家的经验，需要花费大量的人力、物力和财力
 - ❑规则的表达能力有限，很多语言现象无法用简单的规则描述
 - ❑随着规则的增多，规则之间可能存在矛盾和冲突，导致最终无法做出决策 一个句子中既出现了“喜欢”，又出现了“讨厌”，其极性应该是什么呢？

文本表示-向量

- 为了解决基于规则的方法存在的以上诸多问题，**基于机器学习的自然语言处理技术**应运而生
- 最本质的思想是**将文本表示为向量**，其中的每一维代表一个**特征**
- 在进行决策的时候，只要对这些**特征**的相应值进行**加权求和**，就可以得到一个分数用于最终的判断

文本表示-向量

□情感极性识别为例，一种非常简单的将原始文本表示为向量的方法为：

① 令向量 $x = (x_1, x_2, x_3, x_4, \dots)$ 的每一维表示某个词在该文本中出现的次数，如：

□ x_1 表示“我”出现的次数

□ x_2 表示“喜欢”出现的次数

□ x_3 表示“电影”出现的次数

□ x_4 表示“讨厌”出现的次数等

□ 如果某个词在该句中出现未，则相应的维数被设置为0

② 输入向量 x 的大小恰好为整个词表的大小

文本表示-向量

□情感极性识别为例，一种非常简单的将原始文本表示为向量的方法为：

③ 根据每个词对判断情感极性的重要性进行加权

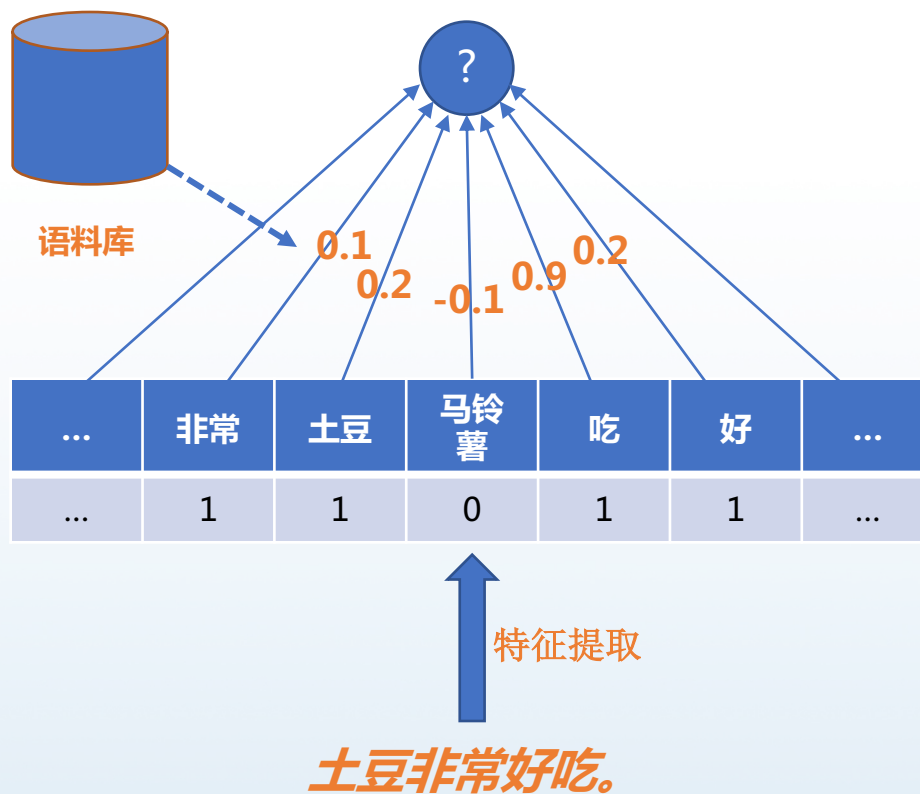
$$x = (x_1, x_2, x_3, x_4, \dots)$$

□ 如

□ “喜欢” (x_2)对应的权重 ω_2 可能比较大

□ “讨厌” (x_4)对应的权重 ω_4 可能比较小(可以为负数)

□ 对于情感极性影响比较小的词，如“我” “电影”等，对应的权重可能会趋近于0



文本表示-向量

□ 向量表示优点

- 可以应用于基于机器学习的方法

- 文本向量表示还可以用于计算两个文本之间的相似

 - 即使用余弦函数等度量函数表示两个向量之间的相似度，并应用于信息检索等任务

文本表示-向量

- 词的独热表示
- 词的分布式表示

词的独热表示

所谓词的独热表示，即使用一个词表大小的向量表示一个词（假设词表为 $\mathbb{V}$ ，则其大小为 $|\mathbb{V}|$ ），然后将词表中的第 i 个词 w_i 表示为向量：

$$e_{w_i} = [0, 0, \dots, \underbrace{1}_{\text{第 } i \text{ 个词}}, \dots, 0] \in \{0, 1\}^{|\mathbb{V}|} \quad (2-1)$$

在该向量中，词表中第 i 个词在第 i 维上被设置为1，其余维均为0。这种表示被称为词的**独热表示**或**独热编码**（One-hot Encoding）。

词的独热表示

我 喜欢 自然 语言 处理。
我 爱 深度 学习。
我 喜欢 机器 学习。



词表: “我” “喜欢”, “自然” “语言”, “处理” “爱”,
“深度” “学习”, “机器”, “。”

词表大小: 10



独热表示:

□ “我”: (1,0,0,0,0,0,0,0,0,0)

□ “语言” (0,0,0,1,0,0,0,0,0,0)

□ “喜欢”: (0,1,0,0,0,0,0,0,0,0)

□ ...

□ “自然”: (0,0,1,0,0,0,0,0,0,0)

词的独热表示

我喜欢自然语言处理。
我爱深度学习。
我喜欢机器学习。

“我”(1,0,0,0,0,0,0,0,0,0) “喜欢”(0,1,0,0,0,0,0,0,0,0)

“自然”(0,0,1,0,0,0,0,0,0,0) “语言”(0,0,0,1,0,0,0,0,0,0)

“处理”(0,0,0,0,1,0,0,0,0,0) “爱”(0,0,0,0,0,1,0,0,0,0)

...

$$x_{\text{我}} = (1, 0, 0, 0, 0, 0, 0, 0, 0, 0)$$

$$x_{\text{喜欢}} = (0, 1, 0, 0, 0, 0, 0, 0, 0, 0)$$

$$x_{\text{爱}} = (0, 0, 0, 0, 0, 1, 0, 0, 0, 0)$$

$$x_{\text{。}} = (0, 0, 0, 0, 0, 0, 0, 0, 0, 1)$$

$$\cos\langle x, y \rangle = \frac{x \cdot y}{\|x\| \cdot \|y\|}$$

$$\cos\langle x_{\text{喜欢}}, x_{\text{爱}} \rangle = 0$$

$$\cos\langle x_{\text{爱}}, x_{\text{处理}} \rangle = 0$$

$$\cos\langle x_{\text{喜欢}}, x_{\text{处理}} \rangle = 0$$

词的独热表示

□缺点

- 无法处理“**多词一义**”的现象

- 不同词使用**完全不同的向量**进行表示

- 导致即使两个词在**语义上很相似**，但是通过余弦函数来度量它们之间的**相似度时值却为0**

- 土豆** : [0, 0, 0, 0, 0, 0, 0, 0, 0, **1**, 0, 0, 0, 0, ...]

- 马铃薯** : [0, 0, 0, 0, 0, 0, 0, 0, 0, **1**, 0, 0, 0, 0, ...]

- 严重的**数据稀疏**问题

- 当训练数据规模有限时，很多语言现象没有被充分地学习到

- 假设在**训练数据**中只见过“**漂亮**”，在**测试数据**中出现了“**美丽**”，虽然它们之间很相似，但是系统仍然无法恰当地对“**美丽**”进行加权

词的独热表示

□增加额外的特征

- 词性特征：名词、动词、形容词

- 前后缀特征：re-、-tion、-er

- 词义特征：WordNet

- 过引入WordNet等语义词典，可以获知“漂亮”和“美丽”是同义词，然后引入它们的共同语义信息作为新的额外特征，从而缓解同义词的独热表示不同的问题

□特征的设计费时费力

□那么有没有办法自动提取特征并设置相应的特征值呢？

词的分布式表示

□ 分布语义假设(Distributional semantic hypothesis)

□ 词的含义可由其上下文词的分布进行表示

□ *You shall know a word by the company it keeps* -- Firth J.R. 1957

he curtains open and the moon shining in on the barely
ars and the cold , close moon " . And neither of the w
rough the night with the moon shining so brightly , it
made in the light of the moon . It all boils down , wr
surely under a crescent moon , thrilled by ice-white
sun , the seasons of the moon ? Home , alone , Jay pla
m is dazzling snow , the moon has risen full and cold
un and the temple of the moon , driving out of the hug
in the dark and now the moon rises , full and amber a
bird on the shape of the moon over the trees in front
But I could n't see the moon or the stars , only the
rning , with a sliver of moon hanging among the stars
they love the sun , the moon and the stars . None of
the light of an enormous moon . The splash of flowing w
man 's first step on the moon ; various exhibits , aer
the inevitable piece of moon rock . Housing The Airsh
oud obscured part of the moon . The Allied guns behind

□ 利用大规模的未标注文本数据，根据每个词的上下文分布对词进行表示

词的独热表示

我 喜欢 自然 语言 处理。
我 爱 深度 学习。
我 喜欢 机器 学习。



词表: “我” “喜欢”, “自然” “语言”, “处理” “爱”,
“深度” “学习”, “机器”, “。”

词表大小: 10



词的独热表示

我 喜欢 自然 语言 处理。
我 爱 深度 学习。
我 喜欢 机器 学习。

词语共现频次表

- 每一项代表一个词 w_i 与另一个词 w_j (上下文)在同一个句子中的共现频次，每个词与自身的共现频次设置为0

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我										
喜欢										
自然										
语言										
处理										
爱										
深度										
学习										
机器										
。										

词的独热表示

我 喜欢 自然 语言 处理。
我 爱 深度 学习。
我 喜欢 机器 学习。

词语共现频次表

- 每一项代表一个词 w_i 与另一个词 w_j (上下文)在同一个句子中的共现频次，每个词与自身的共现频次设置为0

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0									
喜欢										
自然										
语言										
处理										
爱										
深度										
学习										
机器										
。										

词的独热表示

我 喜欢 自然 语言 处理。
我 爱 深度 学习。
我 喜欢 机器 学习。

词语共现频次表

- 每一项代表一个词 w_i 与另一个词 w_j (上下文)在同一个句子中的共现频次，每个词与自身的共现频次设置为0

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2								
喜欢										
自然										
语言										
处理										
爱										
深度										
学习										
机器										
。										

词的独热表示

我 喜欢 自然 语言 处理。
我 爱 深度 学习。
我 喜欢 机器 学习。

词语共现频次表

- 每一项代表一个词 w_i 与另一个词 w_j (上下文)在同一个句子中的共现频次，每个词与自身的共现频次设置为0

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1							
喜欢										
自然										
语言										
处理										
爱										
深度										
学习										
机器										
。										

词的独热表示

我 喜欢 自然 语言 处理。
我 爱 深度 学习。
我 喜欢 机器 学习。

词语共现频次表

- 每一项代表一个词 w_i 与另一个词 w_j (上下文)在同一个句子中的共现频次，每个词与自身的共现频次设置为0

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1						
喜欢										
自然										
语言										
处理										
爱										
深度										
学习										
机器										
。										

词的独热表示

我 喜欢 自然 语言 处理。
我 爱 深度 学习。
我 喜欢 机器 学习。

词语共现频次表

- 每一项代表一个词 w_i 与另一个词 w_j (上下文)在同一个句子中的共现频次，每个词与自身的共现频次设置为0

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1	1					
喜欢										
自然										
语言										
处理										
爱										
深度										
学习										
机器										
。										

词的独热表示

我 喜欢 自然 语言 处理。
我 爱 深度 学习。
我 喜欢 机器 学习。

词语共现频次表

- 每一项代表一个词 w_i 与另一个词 w_j (上下文)在同一个句子中的共现频次，每个词与自身的共现频次设置为0

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1	1	1	1	2	1	3
喜欢										
自然										
语言										
处理										
爱										
深度										
学习										
机器										
。										

词的独热表示

我喜欢自然语言处理。
我爱深度学习。
我喜欢机器学习。

词语共现频次表

- 每一项代表一个词 w_i 与另一个词 w_j (上下文)在同一个句子中的共现频次，每个词与自身的共现频次设置为0

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1	1	1	1	2	1	3
喜欢	2	0	1	1	1	0	0	1	1	2
自然										
语言										
处理										
爱										
深度										
学习										
机器										
。										

词的独热表示

我喜欢自然语言处理。
我爱深度学习。
我喜欢机器学习。

词语共现频次表

- 每一项代表一个词 w_i 与另一个词 w_j (上下文)在同一个句子中的共现频次，每个词与自身的共现频次设置为0

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1	1	1	1	2	1	3
喜欢	2	0	1	1	1	0	0	1	1	2
自然	1	1	0	1	1	0	0	0	0	1
语言										
处理										
爱										
深度										
学习										
机器										
。										

词的独热表示

我喜欢自然语言处理。
我爱深度学习。
我喜欢机器学习。

词语共现频次表

- 每一项代表一个词 w_i 与另一个词 w_j (上下文)在同一个句子中的共现频次，每个词与自身的共现频次设置为0

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1	1	1	1	2	1	3
喜欢	2	0	1	1	1	0	0	1	1	2
自然	1	1	0	1	1	0	0	0	0	1
语言	1	1	1	0	1	1	0	0	0	1
处理	1	1	1	1	0	0	0	0	0	1
爱	1	0	0	0	0	0	1	1	0	1
深度	1	0	0	0	0	1	0	1	0	1
学习	2	1	0	0	0	1	1	0	1	1
机器	1	1	0	0	0	0	0	1	0	1
。	3	2	1	1	1	1	1	2	1	0

词的分布式表示

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1	1	1	1	2	1	3
喜欢	2	0	1	1	1	0	0	1	1	2
自然	1	1	0	1	1	0	0	0	0	1
语言	1	1	1	0	1	1	0	0	0	1
处理	1	1	1	1	0	0	0	0	0	1
爱	1	0	0	0	0	0	1	1	0	1
深度	1	0	0	0	0	1	0	1	0	1
学习	2	1	0	0	0	1	1	0	1	1
机器	1	1	0	0	0	0	0	1	0	1
。	3	2	1	1	1	1	1	2	1	0

□共现矩阵它有以下3个特点：

- 统计的是单词对在给定环境中的共现次数；在一定程度上能表达词间的关系
- 共现频次计数是针对整个语料库而不是一句或一段文档，具有全局统计特征
- 共现矩阵它是对称的

词的分布式表示

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1	1	1	1	2	1	3
喜欢	2	0	1	1	1	0	0	1	1	2
自然	1	1	0	1	1	0	0	0	0	1
语言	1	1	1	0	1	1	0	0	0	1
处理	1	1	1	1	0	0	0	0	0	1
爱	1	0	0	0	0	0	1	1	0	1
深度	1	0	0	0	0	1	0	1	0	1
学习	2	1	0	0	0	1	1	0	1	1
机器	1	1	0	0	0	0	0	1	0	1
。	3	2	1	1	1	1	1	2	1	0

□表中的每一行代表一个词的向量

$$x_{\text{我}} = (0, 2, 1, 1, 1, 1, 1, 2, 1, 3)$$

$$x_{\text{喜欢}} = (2, 0, 1, 1, 1, 0, 0, 1, 1, 2)$$

$$x_{\text{爱}} = (1, 0, 0, 0, 0, 0, 1, 1, 0, 1)$$

$$x_{\text{。}} = (3, 2, 1, 1, 1, 1, 1, 2, 1, 0)$$

词的分布式表示

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1	1	1	1	2	1	3
喜欢	2	0	1	1	1	0	0	1	1	2
自然	1	1	0	1	1	0	0	0	0	1
语言	1	1	1	0	1	1	0	0	0	1
处理	1	1	1	1	0	0	0	0	0	1
爱	1	0	0	0	0	0	1	1	0	1
深度	1	0	0	0	0	1	0	1	0	1
学习	2	1	0	0	0	1	1	0	1	1
机器	1	1	0	0	0	0	0	1	0	1
。	3	2	1	1	1	1	1	2	1	0

□表中的每一行代表一个词的向量

$$\begin{aligned}x_{\text{我}} &= (0, 2, 1, 1, 1, 1, 1, 2, 1, 3) \\x_{\text{喜欢}} &= (2, 0, 1, 1, 1, 0, 0, 1, 1, 2) \\x_{\text{爱}} &= (1, 0, 0, 0, 0, 0, 1, 1, 0, 1) \\x_{\text{。}} &= (3, 2, 1, 1, 1, 1, 1, 2, 1, 0)\end{aligned}$$

□计算两个向量之间的余弦函数，就可以计算两个词的相似度

$$\cos\langle x, y \rangle = \frac{x \cdot y}{\|x\| \cdot \|y\|}$$
$$\begin{aligned}\cos\langle x_{\text{喜欢}}, x_{\text{爱}} \rangle &= 0.69 \\ \cos\langle x_{\text{爱}}, x_{\text{处理}} \rangle &= 0.45 \\ \cos\langle x_{\text{喜欢}}, x_{\text{处理}} \rangle &= 0.75\end{aligned}$$

上下文选择

- 上下文的选择有很多种方式，而选择不同的上下文得到的词向量表示性质会有所不同
- 可以使用词在句子中的一个固定窗口内的词作为其上下文，得到的词表示将更多地反映词的局部性质：具有相似词法、句法属性的词将会具有相似的向量表示。
- 可以使用所在的文档本身作为上下文，得到的词表示将更多地反映词代表的主题信息。

词的分布式表示

直接使用与上下文的共现频次作为词的向量表示，至少存在以下三个问题：

□ 高频词误导计算结果

- 上例 “我” “。” 与其他词的共现频次很高

- 导致实际上可能没有关系的两个词由于都和这些词共现过，从而产生了较高的相似度。

□ 共现频次无法反映词之间的高阶关系。

- 例如，假设词 “A”与 “B”共现过，“B”与 “C”共现过，“C”与 “D”共现过，通过共现频次，只能获知 “A”与 “C”都与 “B”共现过，它们之间存在一定的关系，而 “A”与 “D”这种高阶的关系则无法知晓

□ 仍然存在稀疏性的问题。

- 即向量中仍有大量的值为0，这一点从表中也可以看出。

问题:高频词误导计算结果的问题

□**解决思路**：如果一个词与很多词共现，则降低其权重；反之，如果一个词只与个别词共现，则提高其权重

□**点互信息**：对于词 w 和上下文 c ，其PMI为：

$$\text{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

$P(w, c)$ 、 $P(w)$ 、 $P(c)$ 分别是 w 与 c 的共现概率，以及 w 和 c 分别出现的概率

$$\text{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

- 通过PMI公式计算，如果w和c的共现概率(与频次正相关)较高，但是w或者c出现的概率也较高(高频词)，则最终的PMI值会变小；
- 反之，即便w和c的共现概率不高，但是w或者c出现的概率较低(低频词)，则最终的PMI值也可能会比较大。
- 从而较好地解决高频词误导计算结果的问题。

□通过最大似然估计分别计算相关的概率值。具体公式为：

$$P(w, c) = \frac{C(w, c)}{\sum_{w', c'} C(w', c')} \quad (2-3)$$

$$P(w) = \frac{C(w)}{\sum_{w'} C(w')} = \frac{\sum_{c'} C(w, c')}{\sum_{w'} \sum_{c'} C(w', c')} \quad (2-4)$$

$$P(c) = \frac{C(c)}{\sum_{c'} C(c')} = \frac{\sum_{w'} C(w', c)}{\sum_{w'} \sum_{c'} C(w', c')} \quad (2-5)$$

$C(w, c)$ 表示词 w 和上下文 c 在语料库中出现的次数

$\sum_{c'} C(w, c')$ 为表2-1按行求和

$\sum_{w'} C(w', c)$ 为表2-1按列求和

$\sum_{w'} \sum_{c'} C(w', c')$ 为全部共现频次的和

	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1	1	1	1	2	1	3
喜欢	2	0	1	1	1	0	0	1	1	2
自然	1	1	0	1	1	0	0	0	0	1
语言	1	1	1	0	1	0	0	0	0	1
处理	1	1	1	1	0	0	0	0	0	1
爱	1	0	0	0	0	0	1	1	0	1
深度	1	0	0	0	0	1	0	1	0	1
学习	2	1	0	0	0	1	1	0	1	1
机器	1	1	0	0	0	0	0	1	0	1
。	3	2	1	1	1	1	1	2	1	0

□代入以上3个公式，式(2-2)可以进一步写为：

$$\begin{aligned} \text{PMI}(w, c) &= \log_2 \frac{P(w, c)}{P(w)P(c)} \\ &= \log_2 \frac{\frac{C(w, c)}{\sum_{w', c'} C(w', c')}}{\frac{\sum_{c'} C(w, c')}{\sum_{w'} \sum_{c'} C(w', c')} \frac{\sum_{w'} C(w', c)}{\sum_{w'} \sum_{c'} C(w', c')}} \\ &= \log_2 \frac{C(w, c)}{\frac{\sum_{c'} C(w, c') \sum_{w'} C(w', c)}{\sum_{w'} \sum_{c'} C(w', c')}} \end{aligned} \quad (2-6)$$

□当某个词与上下文之间共现次数较低时，可能会得到负的PMI值

□ 实际应用中通常采用PPMI

$$\text{PPMI}(w, c) = \max(\text{PMI}(w, c), 0) \quad (2-7)$$

```
def pmi(M, positive=True):  
    col_totals = M.sum(axis=0) # 按列求和  
    row_totals = M.sum(axis=1) # 按行求和  
    total = col_totals.sum() # 总频次  
    expected = np.outer(row_totals, col_totals) / total # 获得每个元素的分子  
    M = M / expected  
    with np.errstate(divide='ignore'): # 不显示log(0)的警告:  
        M = np.log(M)  
    M[np.isinf(M)] = 0.0 # 将log(0)置为0  
    if positive:  
        M[M < 0] = 0.0  
    return M
```

```
[[0, 2, 1, 1, 1, 1, 1, 2, 1, 3],  
 [2, 0, 1, 1, 1, 0, 0, 1, 1, 2],  
 [1, 1, 0, 1, 1, 0, 0, 0, 0, 1],  
 [1, 1, 1, 0, 1, 0, 0, 0, 0, 1],  
 [1, 1, 1, 1, 0, 0, 0, 0, 0, 1],  
 [1, 0, 0, 0, 0, 0, 1, 1, 0, 1],  
 [1, 0, 0, 0, 0, 1, 0, 1, 0, 1],  
 [2, 1, 0, 0, 0, 1, 1, 0, 1, 1],  
 [1, 1, 0, 0, 0, 0, 0, 1, 0, 1],  
 [3, 2, 1, 1, 1, 1, 1, 2, 1, 0]]
```

```
[[0.    0.17 0.06 0.06 0.06 0.28 0.28 0.28 0.28 0.28]  
 [0.17 0.    0.43 0.43 0.43 0.    0.    0.    0.65 0.25]  
 [0.06 0.43 0.    1.02 1.02 0.    0.    0.    0.    0.14]  
 [0.06 0.43 1.02 0.    1.02 0.    0.    0.    0.    0.14]  
 [0.06 0.43 1.02 1.02 0.    0.    0.    0.    0.    0.14]  
 [0.28 0.    0.    0.    0.    0.    1.46 0.77 0.    0.36]  
 [0.28 0.    0.    0.    0.    1.46 0.    0.77 0.    0.36]  
 [0.42 0.09 0.    0.    0.    0.9 0.9 0.    0.9 0.    ]  
 [0.28 0.65 0.    0.    0.    0.    0.    0.77 0.    0.36]  
 [0.2 0.17 0.06 0.06 0.06 0.28 0.28 0.28 0.28 0.    ]]
```

奇异值分解

问题：共现频次无法反映词之间高阶关系

□对共现矩阵 M 进行奇异值分解：

$$M = U \Sigma V^{\top} \quad (2-8)$$

$U \in \mathbb{R}^{|\mathbb{V}| \times r}$ 、 $V \in \mathbb{R}^{r \times |\mathbb{C}|}$ 为正交矩阵，满足

$$U^{\top} U = V^{\top} V = I$$

$\Sigma \in \mathbb{R}^{r \times r}$ ：由 r 个奇异值构成的对角矩阵

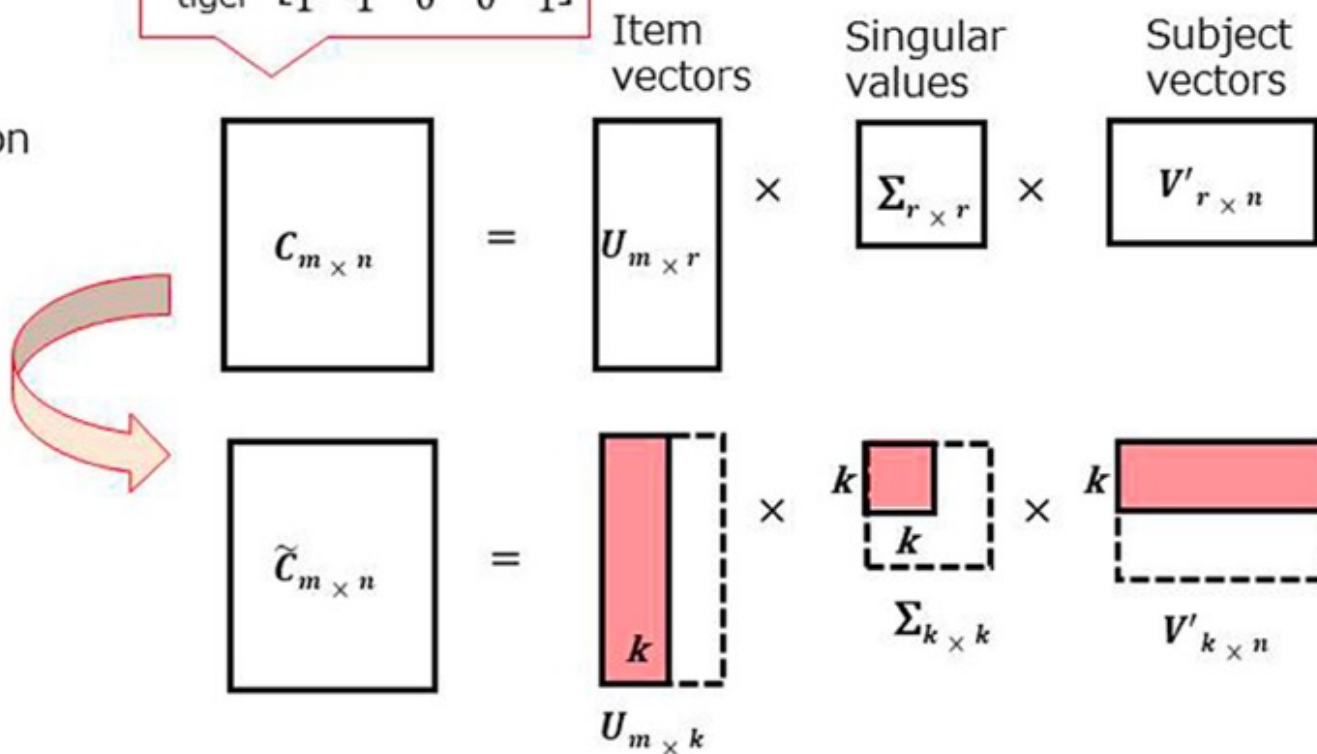
奇异值分解

Item x subject matrix
(ISM)

	S1	S2	S3	S4	S5
dog	1	1	1	1	1
cat	1	1	0	1	0
cow	0	0	1	0	1
lion	0	0	1	1	0
tiger	1	1	0	0	1

Singular decomposition
analysis (SVD)

Reducing dimensions
from r to k



奇异值分解

□ 若在 Σ 中仅保留 k 个($k < r$)最大的奇异值(U 和 V 也只保留相应的维度), 则被称为**截断奇异值分解**。

□ 截断奇异值分解实际上是对矩阵 M 的低秩近似

□ 通过截断奇异值分解所得到的矩阵 U 中的**每一行**, 则为相应词的 k 维向量表示

□ 该向量一般具有**连续**、**低维**和**稠密**的性质。

□ U 的各列相互正交, 可认为词表示的**每一维**表达了该词的一种**独立的“潜在语义”**, 所以这种方法也被称作**潜在语义分析**

□ ΣV^T 的**每一列**也可以作为相应上下文的向量表示

奇异值分解

- 在Python的numpy.linalg库中内置了SVD函数，只需要输入共现矩阵，然后调用相应的函数即可。如：

$$U, s, Vh = \text{np.linalg.svd}(M_{\text{pmi}})$$

- 执行结束后，矩阵U中的每一行为相应词经过奇异值分解后的向量表示。

- 若仅保留前两维，每个词就可以显示为二维平面中的一个点，然后使用下面的代码进行可视化：

```
import matplotlib.pyplot as plt

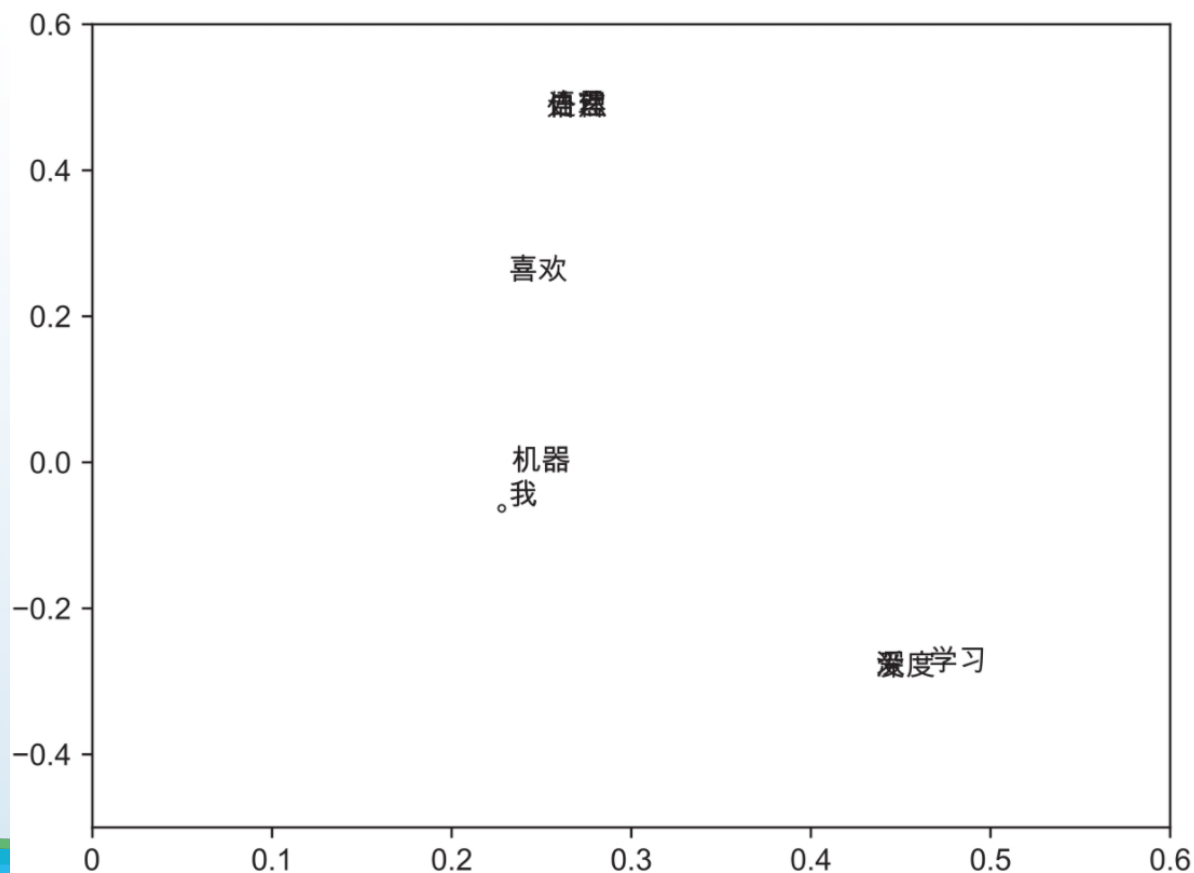
words=["我","喜欢","自然","语言","处理","爱","深度","学习","机器","。"]

for i in range(len(words)):
    plt.text(U[i, 0], U[i, 1], words[i]) # U中的前两维对应二维空间的坐标
```

奇异值分解

截断奇异值分解结果

- 上下文比较相近的词在空间上的距离比较近：“深度” “学习”
- “我” 和 “。” 等高频词则与其他词语距离比较远



□ 信息检索等领域

- 也常通过词与其出现的文档构成“词--文档”共现矩阵
- 也可以通过以上介绍的奇异值分解技术进行降维，并在低维空间(潜在语义空间)内计算词语或者文档之间的相似度，该技术也称潜在语义索引

词的分布式表示

虽然词的分布式表示取得了不错的效果，但是其仍然**存在问题**

- 共现矩阵规模较大时，奇异值分解的运行速度非常慢
- 在原来语料库的基础上增加更多的数据，则需要重新运行奇异值分解算法，代价非常高；
- 只能用于表示比较短的单元，如词或短语等
 - 若待表示的单元比较长，如段落、句子等，由于与其共现的上下文会非常少，则无法获得有效的分布式表示
- 分布式表示一旦训练完成，则无法修改，即无法根据具体的任务调整其表示方式

为了解决这些问题，可引入一种新的词表示方式——**词嵌入表示**

词嵌入表示

词嵌入表示也使用一个连续、低维、稠密的向量来表示词，经常直接简称为词向量，但与分布式表示不同之处在于其赋值方式

□ 词的分布式表示的向量值获取：

- 对语料库进行统计
- 经过点互信息、奇异值分解等变换
- 一旦确定则无法修改

□ 词向量中的向量值：随着目标任务的优化过程自动调整

- 可以将词向量中的向量值看作模型的参数
- 词向量的使用示例将在4.6.2节介绍。

词嵌入表示

- 若目标任务的训练数据比较少，学习合适的词向量难度会比较大
- 利用自然语言文本中所蕴含的自监督学习信号(即词与上下文的共现信息)，先来预训练词向量，往往会获得更好的结果
- 预训练模型的学习和使用，从第5章开始将进行详细的介绍

文本的词袋表示

- 词表示方法(独热表示和分布式表示)
- 如何通过词的表示构成更长文本表示呢？
- 一种最简单的文本表示方法——词袋(Bag-Of-Words , BOW)表示
 - 词袋表示假设：文本中的词语是没有顺序的集合
 - 将文本中的全部词所对应的向量表示(既可以是独热表示，也可以是分布式表示或词向量)相加，即构成了文本的向量表示
 - 词使用独热表示时，文本向量表示的每一维恰好是相应的词在文本中出现的次数

文本的词袋表示

词袋表示优点与缺点

□**优点**: 非常简单、直观

□**缺点**:

① 没有考虑词的顺序信息

□ “张三打李四” 和 “李四打张三”，含义不同，包含词相同、词袋表示一样

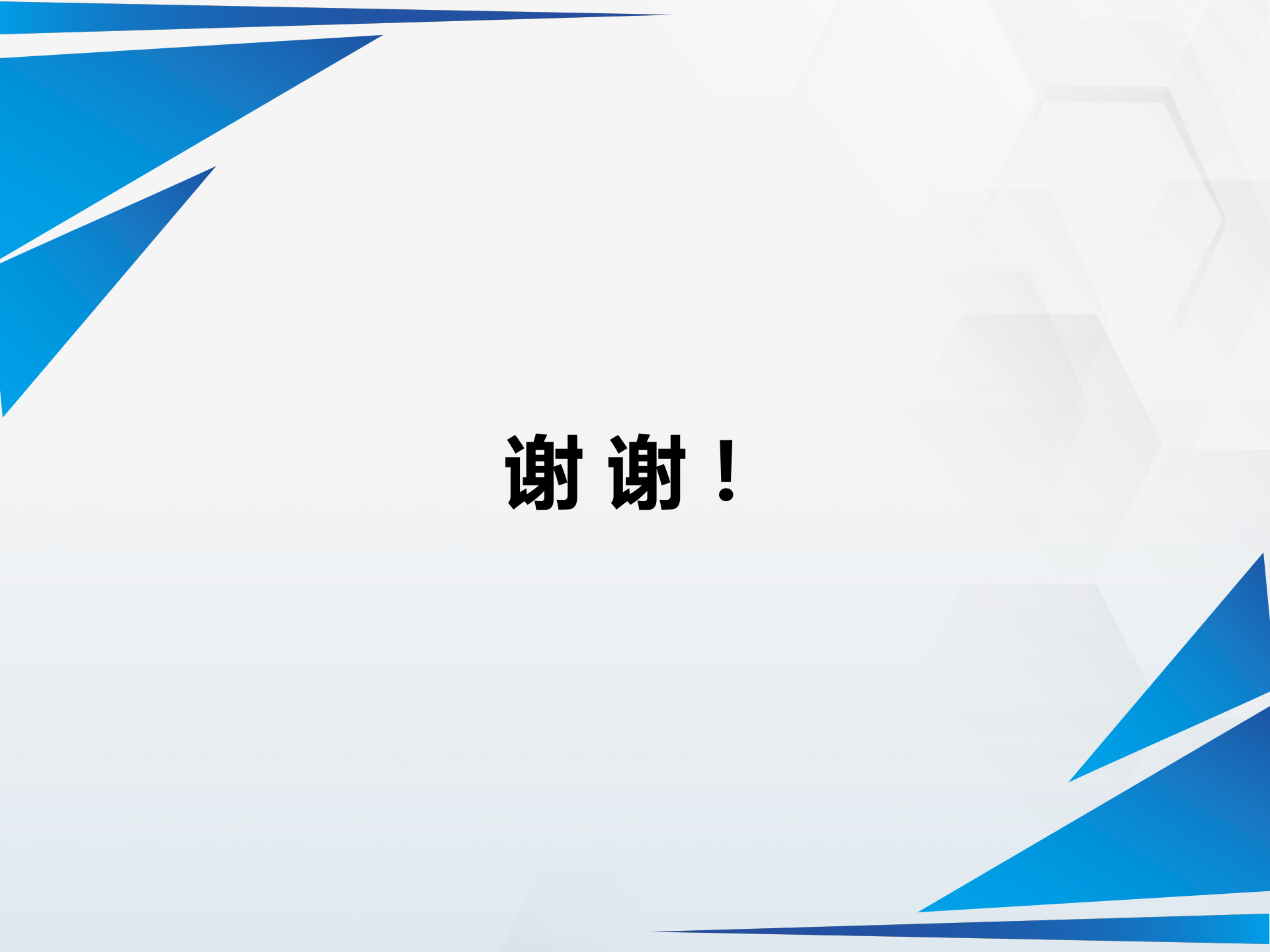
② 无法融入上下文信息

□ 如要表示 “不喜欢”，只能将两个词的向量相加，无法进行更细致的语义操作

□ 可以通过增加词表的方法加以解决，比如引入二元词词表，将 “不+喜欢” 等作为 “词”，然后同时学习二元词的词向量表示

□ 既能部分解决否定词的问题，也能部分解决局部词序的问题，但是随着词表的增大，会引入更严重的数据稀疏问题

□ 深度学习技术的引入为解决这些问题提供了更好的方案，本书后续章节将进行更详细的介绍。



谢谢！